How do I reset my password? → []

answer: []

password passowr

search

neural networK:

cat: [10,20,30,40]

Eucledian distance
400

[100,200,300,400]

I love cat, cat is cat, cat is good

Youtube

cosine use

vector1: [10,20,11,19,60]

vector2: [20,10,9,9,80]

vector3: [9,10,18,29,10]

Eucledian Distance

Cosine Similarity

underRoot((6-4)^2+(7-5)^2)

[4,5]

[6,7]

distance between these points

underRoot(8)
approx: 2.8

Euclidean Distance Formula
Measures straight-line distance between two points
For 2D vectors:
$A = [x_1, y_1]$
$B = [x_2, y_2]$

$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
For 3D vectors:
$A = [x_1, y_1, z_1]$
$B = [x_2, y_2, z_2]$

$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$

Cosine Similarity Formula
Measures angle between two vectors
Formula:
$A = [a_1, a_2, a_3, \ldots, a_n]$
$B = [b_1, b_2, b_3, \ldots, b_n]$

$\text{Cosine Similarity} = (A \cdot B) / (\|A\| \times \|B\|)$
Breaking it down:
Step 1: Dot Product $(A \cdot B)$
$A \cdot B = a_1 \times b_1 + a_2 \times b_2 + a_3 \times b_3 + \ldots + a_n \times b_n$
Step 2: Magnitude of A $(\|A\|)$
$\|A\| = \sqrt{a_1^2 + a_2^2 + a_3^2 + \ldots + a_n^2}$
Step 3: Magnitude of B $(\|B\|)$
$\|B\| = \sqrt{b_1^2 + b_2^2 + b_3^2 + \ldots + b_n^2}$
Step 4: Divide
$\text{Cosine Similarity} = (A \cdot B) / (\|A\| \times \|B\|)$
Result: Value between -1 and 1

1 = Same direction (most similar)
0 = Perpendicular (unrelated)
-1 = Opposite direction (most different)

Note: Higher similarity = more similar

For Vector Embeddings (Text, Images, etc.)
Always use Cosine Similarity
Why? Because embeddings from models (OpenAI, Google, etc.) represent semantic meaning as direction.

Examples:

Text search with embeddings
Semantic similarity
Recommendation systems
Document clustering
Image similarity (using embeddings)
Any RAG application

For Raw Numerical Data
Use Euclidean Distance
Why? Because the actual values matter, not just direction.
Examples:

Physical measurements (height, weight, temperature)
Geographic coordinates (latitude, longitude)
Pixel values in images (RGB colors)
Time series data (stock prices)
Sensor readings

Normalise the embedding

Original vector: [3, 4]

Step 1: Calculate magnitude
magnitude = $\sqrt{3^2 + 4^2}$
= $\sqrt{9 + 16}$
= $\sqrt{25}$
= 5

Step 2: Divide each component
normalized = [3/5, 4/5]
= [0.6, 0.8]

Verify: $\sqrt{0.6^2 + 0.8^2} = \sqrt{0.36 + 0.64} = \sqrt{1} = 1$ ✓

Why do we need Vector Database?

SQL and NoSQL databases are designed to find exact matches based on known values, while vector databases are designed to find similar items based on conceptual meaning.

[0.2,0.9,0.8]

[0.1,0.8,0.4]
[0.2,0.1,0.9]
[0.3,0.1,0.5]
[0.4,0.9,0.8]
[0.6,0.8,0.4]
[0.7,0.8,0.4]

[2,100,150]

[1,4,10]
[2,5,12]
[3,6,18]
[4,200,300]
[6,12,18]
[15,18,19]

sorting karke
koi faiyda
nahi hai

Blinkit

1 Million

sorted
order

o(n)

First Method: Brute Force

aalo ; [0.8,0.2,0.3, ...]

Item	Vector
onion	[0.9, 0.1, 0.0, 0.9, 0.1]
tomato	[0.8, 0.2, 0.0, 0.8, 0.2]
dhaniya	[0.7, 0.1, 0.0, 0.9, 0.0]
nimbu	[0.1, 0.8, 0.1, 0.7, 0.3]
banana	[0.1, 0.9, 0.1, 0.1, 0.9]
protein	[0.0, 0.1, 0.9, 0.1, 0.7]
creatine	[0.0, 0.0, 0.95, 0.0, 0.1]
bcaa	[0.0, 0.0, 0.92, 0.0, 0.1]

bcaa
creatine
protein

Array = [11,8,3,7,19,12] → sorting → Hashing → exact search

target=12

n

log(n)

0(1)

[3,7,8,11,12,13]

[0.0,0.1,0.1,0.9,0.9]

10
11
20
30

The reason a full scan is slow is because it's perfect.
It calculates the distance to every single point and guarantees it finds the absolute closest one.
This is called Exact Nearest Neighbor (ENN) search.

ANN Approach:

Approximate nearest neighbor (ANN) is an algorithm that finds a data point in a data set that's very close to the given query point, but not necessarily the absolute closest one.

It sacrifices a tiny amount of accuracy (it might return 9 of the top 10, plus the 11th) for a massive gain in speed (often orders of magnitude faster).

First Method: Clustering /Inverted File Index - IVF

Phase 1: Indexing (The one-time setup)
Find the "Centers": The database first looks at all of your million vectors and uses an algorithm (like k-means clustering) to find, let's say, 100 "center points" or centroids. Each centroid is itself a vector that represents the average of a region in your data. Think of these as the signs for the sections in our library: "Fiction," "Science," etc.

Assign to Clusters: The database then goes through every single one of your million vectors and figures out which of the 100 centroids it's closest to. It then puts a "pointer" to that vector into a list for that centroid.

Searching (When you make a query)
Now you come with your query vector for "light blue summer shirt."
Find the Closest Section(s): The database does NOT compare your query to all one million vectors. Instead, it just compares it to the 100 centroids. This is super fast. It finds that your query is closest to, say, "Centroid #42" (the "men's casual shirts" section).

Search Only Within That Section: Now, the database knows your best matches are probably in the list for Centroid #42.
So it only searches through the 10,000 vectors in that one cluster.

average

1 million vector = 100 cluster

1oo centroid

5k 2k 20k 2k

1 2. 3. 4.

100 (centroid)

30k

average

100 centroid average

1 million vector

100 centroid

10k 10k

100+10k

10 most nearest

pant = [10,3]

o(n)

top 10 nearest

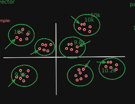
7 centroid

onion = [2,3]
orange = [4,5]

1 lakh vector

50k

Example:



The Goal
You have millions of vectors scattered in high-dimensional space. You want to group them into, say, 1,000 clusters where vectors in the same cluster are "close" to each other.

The K-Means Algorithm - Step by Step
Step 1: Initialize Centroids Randomly
Pick 1,000 random vectors from your dataset as initial "centroids" (cluster centers).
These are just starting guesses - they'll move around as the algorithm runs.
Think of it like: You're organizing a party with 1,000 tables. You randomly place 1,000 chairs as "table centers" in the room.

Step 2: Assignment Phase
For every single vector in your dataset:

Calculate distance to all 1,000 centroids
Assign the vector to the nearest centroid

This is like: Every guest walks to the nearest chair and stands there.
After this step: Each centroid now has a group of vectors assigned to it. Some centroids might have 5,000 vectors, others might have only 200.

Step 3: Update Phase
For each centroid:

Take all the vectors assigned to it
Calculate the mean/average of all those vectors
Move the centroid to that average position

This is like: You look at all the guests standing around each chair, find the "center of mass" of that crowd, and move the chair there.
Why this matters: The centroid moves closer to where most of its assigned vectors actually are.

Step 4: Repeat Until Convergence

Go back to Step 2. Reassign all vectors to their nearest centroids (which have now moved).
Then Step 3 again - recalculate centroids based on new assignments.
Keep repeating until one of these happens:

Centroids stop moving (or move very little)
Assignments stop changing
You hit a maximum number of iterations (like 100)

Drawback

1. Training Cost

Must run k-means on entire dataset upfront
For 100M vectors, this takes hours/days
Need to retrain if data distribution changes significantly

2. The "Border Problem"

Your query might be near the edge of a cluster
Relevant vectors could be in the neighboring cluster
You miss them unless you increase nprobe (search more clusters)
Higher nprobe → slower queries, defeating the purpose

3. Poor for Dynamic Data

Adding new vectors is awkward - which cluster do they go to?
Deleting vectors leaves clusters imbalanced
Eventually need to retrain/rebalance

4: Quality Depends on K

Wrong choice of K (number of clusters) hurts performance
Too few → slow queries
Too many → miss relevant results
No automatic way to pick optimal K

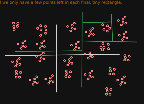
Second Method: The Decision Tree Method (Binary Space Partitioning)

if we just took the whole space and chopped it in half?
And then chopped those halves in half? And so on.

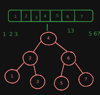
First Split: We pick a dimension (e.g., the x-axis) and find the middle point (the median). We draw a vertical line there. All points to the left go into the "left" branch of our tree, and all points to the right go into the "right" branch.

Second Split: Now we look at the two new groups. For each group, we pick the other dimension (the y-axis) and split it in half with a horizontal line.

Keep Splitting: We keep repeating this process, alternating which dimension we split on, until we only have a few points left in each final, tiny rectangle.



→ 6 1 3 5 7 2 4



Point	Vector [X, Y]	Point	Vector [X, Y]	Point	Vector [X, Y]
A	[2, 7]	F	[9, 4]	K	[14, 15]
B	[3, 16]	G	[10, 19]	L	[15, 3]
C	[4, 4]	H	[11, 3]	M	[17, 11]
D	[5, 11]	I	[12, 13]	N	[18, 5]
E	[7, 14]	J	[19, 10]	O	[19, 9]

A B C D E F G

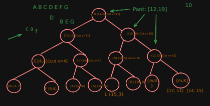
D

B E G

c a f

Pant: [12,19]

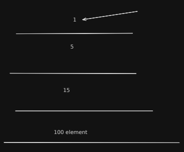
10



Search 3 NN: Q = [13, 8]

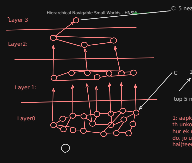
work

log (n)



Hierarchical Navigable Small Worlds - HNSW

C: 5 nearest



C

12 nearest

top 5 nearest

1: aapke pass jitne bhi vector th unko meine node bola hai, hur ek node usko connect karwa do, jo uske sabse jaada kareeb hai(teen)

Core Idea
Graph-based vector search using hierarchical layers for fast approximate nearest neighbor search.
Structure

Multiple layers (0 to ~log N)
Layer 0: All vectors, each connected to M neighbors (~16)
Upper layers: Exponentially fewer vectors (50% promotion probability per layer)
Same vector exists at multiple layers with different connections per layer

Building the Index
Time complexity: O(N log N)
For each new vector:

Randomly assign max_level (probability 1/2 per level)
Navigate from top layer to find entry points
Insert at each layer (max_level down to 0)
Connect to M nearest neighbors at each layer
Neighbors connect back (bidirectional)
Prune if neighbors exceed M connections (keep closest M)

Query Process
Time complexity: O(log N)

Start at top layer (entry point)
Greedy navigation: jump to closest neighbor
Drop down layers progressively
At Layer 0: beam search with efSearch parameter
Return top K results

efSearch: Controls exploration thoroughness at Layer 0 (typically 50-100)

Key Parameters

M = 16: Connections per node
mL = 2: Layer promotion probability (1/2^level)
efSearch: Search beam width at query time

Advantages
No training phase (unlike IVF k-means)
O(log N) insertion - great for dynamic data
No border problem - smooth navigation
Better accuracy-speed tradeoff
Easy add/delete operations

Disadvantages

Higher memory (stores graph structure)
Slower build time than simple clustering

What is efSearch?
efSearch = "exploration factor at search time"
It's a parameter that controls how thoroughly you explore at Layer 0.

How it works at Layer 0:
Remember you're doing beam search - maintaining a pool of promising candidates
efSearch = 50 means:
"Keep a dynamic list of the 50 most promising vectors as you navigate"

Process:

You arrive at Layer 0 from upper layers (start at some vector)
Add it to your candidate list
Check its neighbors, add them to list
Keep only the best 50 closest to query (this is efSearch)
From these 50, pick the closest unvisited one
Explore its neighbors, add to list
Again keep only best 50
Repeat until all top 50 candidates have been explored
Return top 10 from your final list

Example:

You might visit 200 vectors total during navigation
But you only keep the best 50 in your active pool at any time
From those 50, you return the top 10 as your answer

Higher efSearch = more thorough search = slower but more accurate

Approach 4: The Compression Method (Product Quantization - PQ)

A single vector using 1536 dimensions of 32-bit floating-point numbers takes up 1536 * 4 bytes = 6,144 bytes. If you have a billion vectors, that's 6.1 Terabytes of RAM! That's incredibly expensive.

[0.2,0.9, ...]

Storing color: []

4 bytes

0 - 255

12GB

42

[1536]

[12,31,122] = Pink

[42]

[128] [128] [128] ...

3 bytes

[16,11, ...201]

12 byte

It chops the big vector into smaller segments. For example,
a 16-dimension vector could be chopped into 4 segments of 4 dimensions each.
[d1, d2, ..., d16] -> [chunk1], [chunk2], ..., [chunk4]

V = [1.1, 2.3, 0.9, 3.4, 8.8, 7.6, 9.1, 6.5, 4.2, 5.5, 3.9, 5.1, 9.9, 8.1, 7.7, 9.3]

total size = 16*4 = 64 bytes

v_chunk1 = [2.1, 1.3, 1.9, 3.1]

Step A: Split V into chunks.
v_chunk1 = [1.1, 2.3, 0.9, 3.4] → [10]
v_chunk2 = [8.8, 7.6, 9.1, 6.5] → [23]
v_chunk3 = [4.2, 5.5, 3.9, 5.1] → [123]
v_chunk4 = [9.9, 8.1, 7.7, 9.3] [16]

[10,23,123,16]

Step B: Create the Codebooks (k cluster/centroid)
Chunk 1 (dims 1-4): We take all the first chunks and look at Codebook 1.
Chunk 2 (dims 5-8): We do the same for the second chunks and look at Codebook 2.
Chunk 3 (dims 9-12): This gives us Codebook 3.
Chunk 4 (dims 13-16): This gives us Codebook 4.

→ codeBook1 look like this

1 billio record: chunk1: 256 cluster(256 centroid)

0 : [1.2,3.6,4.7,2.1]
1 : [1.1,3.2,4.5,2.8]
2 : [2.2,3.6,5.7,1.1]
3 : [1.1,2.4,1.0,3.3]
.
.
.
255 : [4.2,3.6,1.7,2.1]

How to search a query:

Q = [1.5, 2.1, 0.8, 3.9, 8.1, 7.9, 9.0, 6.6, 4.8, 5.2, 4.1, 5.0, 9.5, 8.3, 7.9, 9.1]

Split Q into chunks, just like we did for V:
q_chunk1 = [1.5, 2.1, 0.8, 3.9]
q_chunk2 = [8.1, 7.9, 9.0, 6.6] → codeBook1: 256 [1.3,2.3,............]
q_chunk3 = [4.8, 5.2, 4.1, 5.0]
q_chunk4 = [9.5, 8.3, 7.9, 9.1]

codebook2:256
codebook3:256
codebook4: 256

v1 = [10,23,123,16]
v2: [11,13,11,16]

Build the Distance Look-up Tables

1.3 +6.1+ 1.5+9.1 = 18

For Chunk 1: We calculate the squared Euclidean distance from q_chunk1 to every single one of the 256 centroids in Codebook 1.
dist_table1[0] = distance(q_chunk1, Codebook1_centroid0)²
dist_table1[1] = distance(q_chunk1, Codebook1_centroid1)²
...
dist_table1[255] = distance(q_chunk1, Codebook1_centroid255)²
For Chunk 2: We do the same, comparing q_chunk2 to every centroid in Codebook 2, creating dist_table2.
For Chunk 3 & 4: We repeat the process to create dist_table3 and dist_table4.

Result: We now have 4 tables, each with 256 pre-calculated squared distance values.
This took 4 * 256 = 1024 distance calculations in total.

Which Method is "Best"? A Comparison
Let's compare the methods we've discussed across the three critical dimensions.

Method	Speed (Query Latency)	Accuracy (Recall)	Memory Usage
HNSW	Highest	Highest	Highest
IVFPQ (Hybrid)	High	High (with re-ranking)	Lowest by Far
IVF (Standalone)	Medium	Medium	Low
Trees (e.g., KD-Tree)	Low (in high dimensions)	Low	Medium

Database / Library	Category	Key Feature / Philosophy	Primary Indexing Technique(s) Used
Milvus	Native	Open-source, highly scalable, MLOps-focused. Decoupled storage & compute.	Highly Flexible: HNSW, IVFPQ, IVFFlat, and others.
Pinecone	Native	Managed service (SaaS), ease of use, performance-focused.	HNSW
Weaviate	Native	Open-source, GraphQL API, connects vectors in a knowledge graph.	HNSW
Qdrant	Native	Open-source, performance-focused with advanced filtering. Written in Rust.	HNSW with custom quantization.
PostgreSQL (pgvector)	Integrated	Keep your structured data and vectors together in a trusted relational DB.	IVFFlat and HNSW
Elasticsearch	Integrated	Combine traditional keyword search with semantic vector search in one query.	HNSW
Redis	Integrated	In-memory speed, combining vector search with other Redis data structures.	HNSW
Faiss (by Meta)	Library	The foundational C++/Python toolkit. Not a DB, but powers many others.	The Reference: HNSW, IVFPQ, and many more.

what data is actually stored:

[0.98, 0.23, -0.11, ...]

[0.91, 0.20, -0.12, ...]

```
my_id = "product_456"
my_vector = [0.98, 0.23, -0.11, ...]
my_metadata = {
    "product_name": "Red Running Shoes",
    "price": 89.99
}
```

id: unique
vector
metaData: {
link: "hdsjhkbfjk"

word
Sentence
vector